Microsoft Azure

Spec2Cloud et Citadel

Global AI community

19 mars 2026

Présentation Spec2Cloud et Citadel

https://lnkd.in/eUe_bBrz

Agenda

- Spec2Cloud
- Citadel

Agenda

- **Spec2Cloud**
- Citadel

What is Spec2Cloud?

An AI-powered development workflow that transforms high-level product ideas into production-ready applications deployed on Azure, using specialized GitHub Copilot agents.

Greenfield

Transform product ideas into deployed apps through spec-driven development

Brownfield

Reverse engineer existing codebases into documentation, then modernize

Agent-Driven

Specialized AI agents (PM, Dev Lead, Dev, Azure) collaborate end-to-end

The New Blueprint for Building Software

Spec2Cloud
Spec-Driven Development with GitHub Copilot and Azure

Pre-built code | Code created w/ specs | Specs | Agent.md[1]

What is it?

A **Spec** is a structured, machine-readable blueprint that defines what a solution should do (purpose, inputs, outputs) to enable consistent automation, validation, and reuse

An **agent.md** file is like a "readme" / a manual designed for machine-readable instructions that help **coding agents** work effectively in your repo.

	Current status (Crawl)	Spec enhanced (Walk)	Full Spec ahead (Run)
Specs			...
Coded solution (e.g., accelerator)			Evals, Add. DB
Customer requirements			
Explanation	Solutions delivered as rigid codebases that rarely match customer needs - require manual rework	A standardized core codebase is provided, then tailored to customer requirements through editable specs.	Entire solution "automatically" composed from specs, eliminating manual coding & adjustments

Advantages

- Opinionated, MSFT-aligned stack
- Extensible, with cross- & upsell
- Simplified mainten., change mgmt.
- Consistent delivery across ecosystem
- Faster time-to-PoC and time-to-prod

What is Spec-Driven Development?

Spec2Cloud
Spec-Driven Development with GitHub Copilot and Azure

Methodology that emphasizes creating clear, structured specifications before writing any code

From

Code-Centric Delivery

Specs are often minimal or informal, with emphasis on writing and deploying code. Prompts used with AI coding agents are frequently undocumented and forgotten.

Human-Heavy Development

Engineers manually translate business needs directly into code.

Documentation as an Afterthought

Written mainly for compliance or after-the-fact reference.

Value in Execution

Competitive advantage came from how fast you could build and deploy.

Shift

To

Spec-Centric Delivery

Specs are the primary control surface; code is generated by AI. Reduces rework and ensures alignment across teams.

AI-Augmented Development

Agentic DevOps handles the full software lifecycle by using domain specific MCP tools.

Documentation as Communication

Specs serve as tools for aligning stakeholders, preserving decision rationale, and providing context for future maintenance.

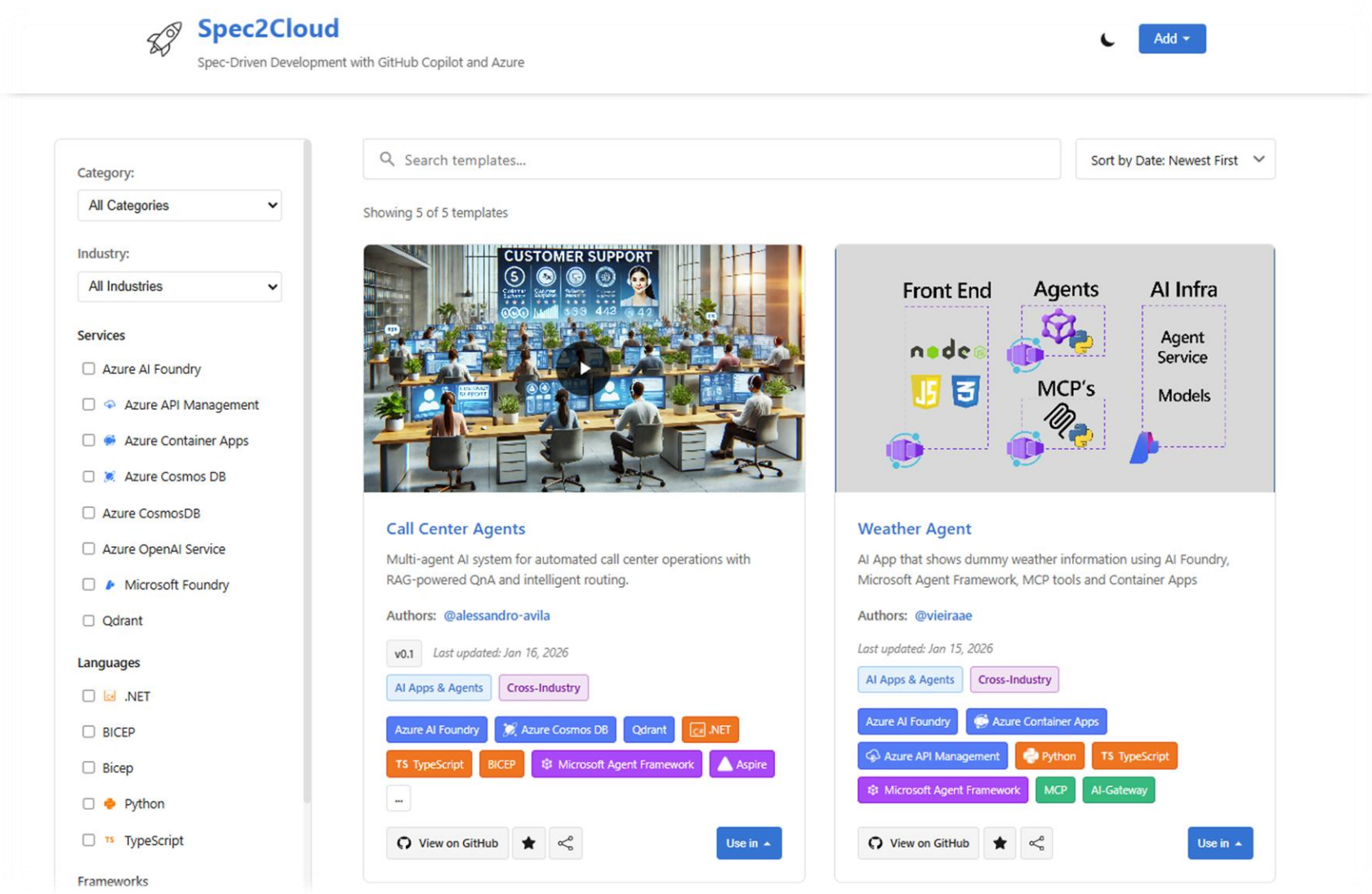
Value in Design

Competitive advantage comes from reusable, high-quality Spec Templates that accelerate Proof of Value and scale.

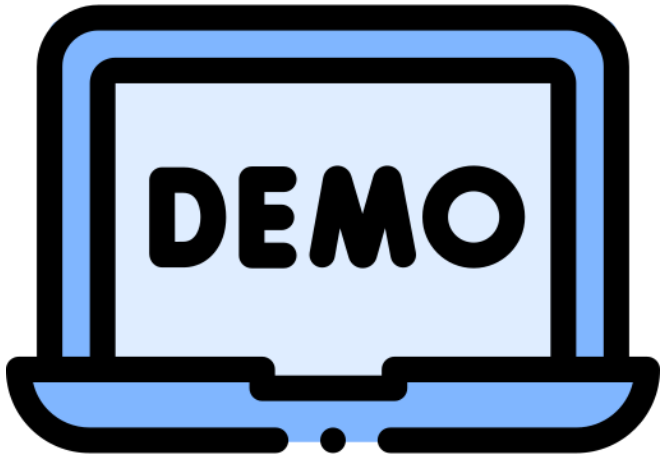
https://azure-samples.github.io/Spec2Cloud/

Spec2Cloud

Spec-Driven Development with GitHub Copilot and Azure



Spec2Cloud demo



Demo App: Wikipedia Quiz

AI-powered quiz generator that creates custom multiple-choice quizzes from Wikipedia articles using Azure OpenAI GPT-4.1 and a LangGraph agentic pipeline.

[sjuratov/wikipedia-quiz-app at feature/react-fastapi-langgraph-azure-openai]

Characteristics

- **Any Topic**: Generate quizzes on virtually any Wikipedia topic
- **AI-Powered**: Intelligent question generation using Microsoft Foundry gpt-4.1 model
- **Customizable**: Choose number of questions (3-20)
- **Instant Feedback**: Immediate results with detailed explanations
- **Source Verification**: Direct links to Wikipedia sources for each answer
- **Smart Caching**: Reduced API calls with intelligent caching
- **Responsive**: Works on desktop and mobile devices

How it works

1. **User Input**: Enter any Wikipedia topic and select number of questions.
2. **Content Retrieval**: System fetches Wikipedia article via API.
3. **AI Generation**: Azure OpenAI generates intelligent multiple-choice questions.
4. **Quality Validation**: Questions validated for clarity and accuracy.
5. **Interactive Quiz**: User answers questions in clean interface.
6. **Instant Results**: Immediate feedback with score and answer verification links.

How it works

- This is an AI-powered quiz generator that creates custom multiple-choice quizzes from Wikipedia articles. You enter any topic, and the system fetches the relevant Wikipedia article, uses an LLM to generate questions, validates them, and serves an interactive quiz through a React frontend

- The backend is a **FastAPI** application (python web framework) served by **Uvicorn** (Asynchronous Server Gateway interface). It uses **langchain-openai** with AzureChatOpenAI to connect to Azure OpenAI (**GPT-4 deployment**), and **LangGraph** to orchestrate the quiz generation pipeline as a directed acyclic graph.

- Frontend (**TypeScript**). A single-page application built with **React.**

- The project uses **LangGraph** to implement a sequential 4-node pipeline (not a multi-agent/collaborative system, but rather a single-agent pipeline with distinct processing stages):
  - Node 1: **resolve_topic** (Topic Resolution)
  - Node 2: **extract_content** (Content Extraction)
  - Node 3: **generate_questions** (Question Generation)
  - Node 4: **validate_questions** (Quality Validation)

Project structure


```
├── backend/            # FastAPI backend
│   ├── app/
│   │   ├── agents/     # LangGraph quiz generation pipeline
│   │   ├── api/        # REST API endpoints
│   │   ├── models/     # Pydantic data models
│   │   ├── services/   # Wikipedia client
│   │   └── utils/      # Caching utilities
│   └── requirements.txt
├── frontend/           # React frontend
│   ├── src/
│   │   ├── components/ # React components
│   │   ├── services/   # API client
│   │   └── types/      # TypeScript types
│   └── package.json
├── specs/              # Product & technical specs
│   ├── prd.md         # Product requirements
│   ├── features/      # Feature specifications
│   └── adr/           # Architecture decisions
└── docs/              # Documentation
```

Specs/prd.md

The **PM Agent** generated this **Product Requirements Document** from a high-level idea.

Creates PRD & FRDs from ideas. Asks clarifying questions, identifies business goals.

[wikipedia-quiz-app/specs/prd.md at feature/react-fastapi-langgraph-azure-openai · sjuratov/wikipedia-quiz-app]

Product Requirements Document: Wikipedia Quiz Application

Version: 1.0
Date: December 11, 2025
Status: Draft

---

1. Product Overview

1.1 Vision

Create an intelligent, browser-based quiz application that leverages Wikipedia's vast knowledge base to generate custom educational quizzes on any topic. The application empowers users to test and expand their knowledge through AI-generated multiple-choice questions tailored to their interests.

1.2 Problem Statement

Students, educators, and lifelong learners need an accessible way to test their knowledge on specific topics without manually creating quizzes or relying on pre-made question banks with limited subject coverage. Existing solutions lack flexibility in topic selection and question quantity customization.

1.3 Value Proposition

- **On-Demand Learning:** Generate quizzes on virtually any topic covered by Wikipedia
- **Customizable Difficulty:** Control the depth and breadth of questions through topic specificity
- **Zero Preparation:** No need to manually create questions or source content
- **Instant Feedback:** Immediate validation of knowledge and identification of learning gaps

specs/features/answer-verification-links.md

Spec2Cloud
Spec-Driven Development with GitHub Copilot and Azure

A detailed **Feature Requirements Document.**

[wikipedia-quiz-app/specs/features/answer-verification-links.md at feature/react-fastapi-langgraph-azure-openai · sjuratov/wikipedia-quiz-app]

Feature Requirements Document: Answer Verification Links

Version: 1.0
Date: January 5, 2026
Status: Approved
Feature Type: Enhancement
Priority: Medium

---

1. Feature Overview

1.1 Summary

Add reference links to quiz results that allow users to verify correct answers by linking to the specific Wikipedia article section where the answer can be found.

1.2 Problem Statement

Currently, when users review quiz results, they can see which answers were correct or incorrect, but they have no way to verify the accuracy of the correct answer or learn more about the topic. Users must manually search Wikipedia to validate information, creating friction in the learning experience.

1.3 Solution

Add clickable "Verify" reference links next to each correct answer in the quiz results view. These links will direct users to the specific section of the Wikipedia article that contains the information supporting the correct answer.

1.4 User Value

- **Trust Building**: Users can independently verify that correct answers are accurate
- **Extended Learning**: Easy access to deeper information on the topic
- **Reduced Friction**: One-click access to source material without manual searching

specs/adr/

Spec2Cloud

Spec-Driven Development with GitHub Copilot and Azure

Architecture Decision Records (frontend framework choice, LLM framework, caching, deployment).

[wikipedia-quiz-app/specs/adr at feature/react-fastapi-langgraph-azure-openai · sjuratov/wikipedia-quiz-app]

ADR 001: Frontend Framework Selection for Wikipedia Quiz Application

Date: December 11, 2025
Status: Accepted
Deciders: Development Team
Technical Story: Wikipedia Quiz Application Frontend Architecture

Context

We need to select a frontend framework for building the Wikipedia Quiz Application, a browser-based quiz platform that generates custom quizzes from Wikipedia content. The application requires:

Functional Requirements

- User input form for topic and question count
- Dynamic quiz generation and display
- Multiple-choice question presentation
- Real-time answer validation
- Results page with scoring and feedback
- Responsive design for mobile and desktop

ADR 002: LLM Provider and Agent Framework Selection

Status

Accepted

Context

The Wikipedia Quiz Application requires an LLM-powered agentic system to generate high-quality quiz questions from Wikipedia content. The system implements a 4-stage pipeline:

1. **Topic Resolution** - Disambiguate user queries and find appropriate Wikipedia articles
2. **Content Extraction** - Retrieve and parse relevant Wikipedia content
3. **Question Generation** - Create multiple-choice questions from the content
4. **Quality Validation** - Ensure questions meet quality standards

Performance Requirements

- Total pipeline execution: <10 seconds
- LLM processing time: <6 seconds
- Support for real-time user interactions

ADR 003: Caching Strategy

Status

Accepted

Context

The Wikipedia Quiz Application needs to cache multiple types of data with different lifetimes:

- **Wikipedia content**: Article text fetched from Wikipedia API (suggested TTL: 7 days)
- **Generated quizzes**: AI-generated quiz questions from articles (suggested TTL: 24 hours)
- **Quiz sessions**: Active user quiz sessions and scores (suggested TTL: 1 hour)

Caching reduces API calls to Wikipedia, minimizes expensive LLM operations, and improves response times. However, performance gains against operational complexity for a demo/learning application.

ADR 004: Deployment Architecture

Status

Accepted

Context

The Wikipedia Quiz Application consists of three main components:

- **FastAPI Backend**: Python-based REST API serving quiz logic
- **React Frontend**: Static JavaScript SPA for user interface
- **Caching Layer**: In-memory cache (see ADR 003)

We need to determine the deployment architecture that balances simplicity, cost, and developer experience for a demo/learning application.

Decision Drivers

- **Simplicity**: Easy to deploy and maintain
- **Developer Experience**: Quick local setup and iteration
- **Cost**: Minimal operational expenses
- **Deployment Speed**: Fast to get running
- **Learning Value**: Demonstrates practical deployment patterns

specs/IMPLEMENTATION_PLAN.md

Spec2Cloud
Spec-Driven Development with GitHub Copilot and Azure

The **Dev Agent's** technical breakdown.

[wikipedia-quiz-app/specs/IMPLEMENTATION_PLAN.md at feature/react-fastapi-langgraph-azure-openai · sjuratov/wikipedia-quiz-app]

Wikipedia Quiz Application - Implementation Plan

Version: 1.0
Date: December 11, 2025
Status: Active

Executive Summary

This document provides a comprehensive implementation plan for the Wikipedia Quiz Application - an AI-powered quiz generation platform that creates custom multiple-choice quizzes from Wikipedia content.

Technology Stack:

- **Frontend:** React + Vite + TypeScript
- **Backend:** FastAPI (Python)
- **AI/Agents:** Azure OpenAI + LangGraph
- **Caching:** In-memory (Python dict with TTL)
- **Deployment:** Single Docker container (FastAPI serving static files)

backend/app/agents/quiz_generator.py

The **LangGraph** (agent orchestration) pipeline as the key implementation.

[wikipedia-quiz-app/backend/app/agents/quiz_generator.py at feature/react-fastapi-langgraph-azure-openai · sjuratov/wikipedia-quiz-app]

```python
"""Main FastAPI application entry point."""
import os
from pathlib import Path
from contextlib import asynccontextmanager
from dotenv import load_dotenv
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse

# Load environment variables from .env file
env_path = Path(__file__).parent.parent / ".env"
load_dotenv(dotenv_path=env_path)

from app.api import quiz
from app.utils.cache import cache_manager


@asynccontextmanager
async def lifespan(app: FastAPI):
    """Application lifespan events."""
    # Startup
    print("🚀 Starting Wikipedia Quiz Application...")
    yield
    # Shutdown
    print("👋 Shutting down...")


app = FastAPI(
    title="Wikipedia Quiz Application",
    description="AI-powered quiz generation from Wikipedia content",
```

Demo App: Wikipedia Quiz

AI-powered quiz generator that creates custom multiple-choice quizzes from Wikipedia articles using Azure OpenAI GPT-4.1 and a LangGraph agentic pipeline.

Any Topic
Search Wikipedia for any subject

AI-Powered
GPT-4.1 generates intelligent MCQs

Instant Feedback
Score + detailed explanations

1. User enters topic
2. Wikipedia API fetch
3. GPT-4.1 generates Qs
4. Validation pipeline
5. Interactive quiz UI

Greenfield Workflow

Idea → Specs → Code → Cloud

1	2	3	4	5	6
/prd	/frd	/generate-agents	/plan	/implement	/deploy
PRD	**FRDs**	**Standards**	**Tasks**	**Code**	**Deploy**
@pm	@pm	@dev-lead	@dev	@dev	@azure
Product Requirements	Feature Requirements	AGENTS.md	Implementation Plan	Build the app	IaC + CI/CD

Architecture

Frontend

- React + TypeScript
- Vite dev server
- Modern CSS
- Responsive UI

→ **REST API**

Backend

- FastAPI (Python)
- LangGraph Agent
- Wikipedia Client
- Caching Layer

→ **Entra ID**

Azure

- AI Foundry
- OpenAI GPT-4.1
- Entra ID Auth

Spec2Cloud demo video

Spec2Cloud

Spec-Driven Development with GitHub Copilot and Azure

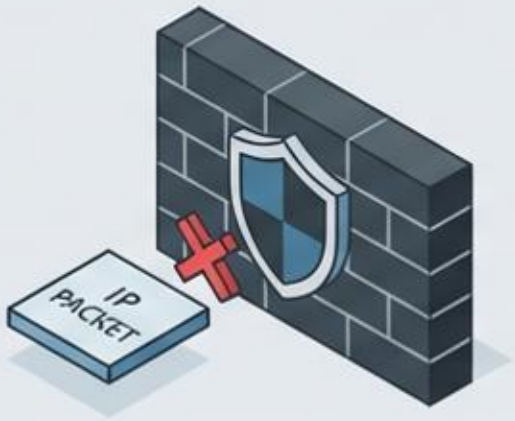
Spec2Cloud resources

Agenda

- Spec2Cloud
- **Citadel**

Traditional security is blind to agentic intent

The Old Perimeter



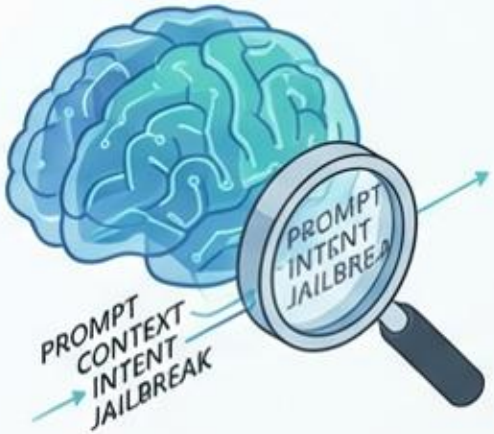
Network vs. Semantic

WAFs inspect packets (IPs, headers). They cannot read prompts.

Static Policy

Allow-lists fail against autonomous decision-making.

The New Threat Surface



Semantic Inspection

Citadel inspects intent, context, and jailbreak attempts.

Dynamic Identity

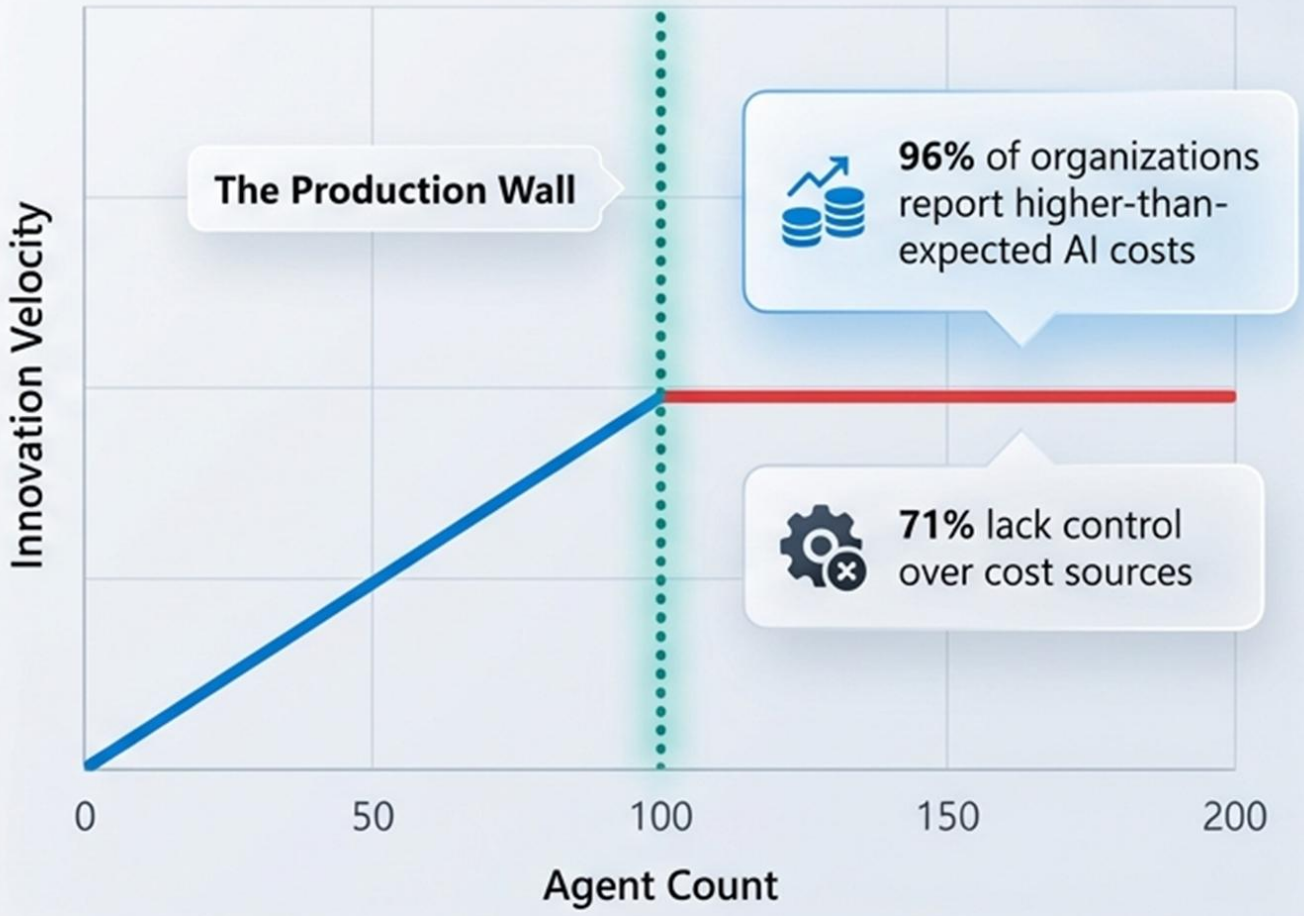
Agents require distinct Workload Identities (Entra ID), not generic service accounts.

Why scaling agents breaks current infrastructure?

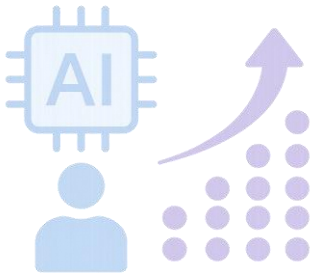
The Governance-Velocity Paradox

Innovating quickly vs. controlling risk creates an operational deadlock.

- **Shadow AI**: Unknown agents consuming resources without oversight.
- **Operational Friction:** Manual risk assessments cannot keep pace with automated deployments.



The AI Governance Imperative



AI is transformative

AI Agents are moving from chat to act and moving from few to 1000s

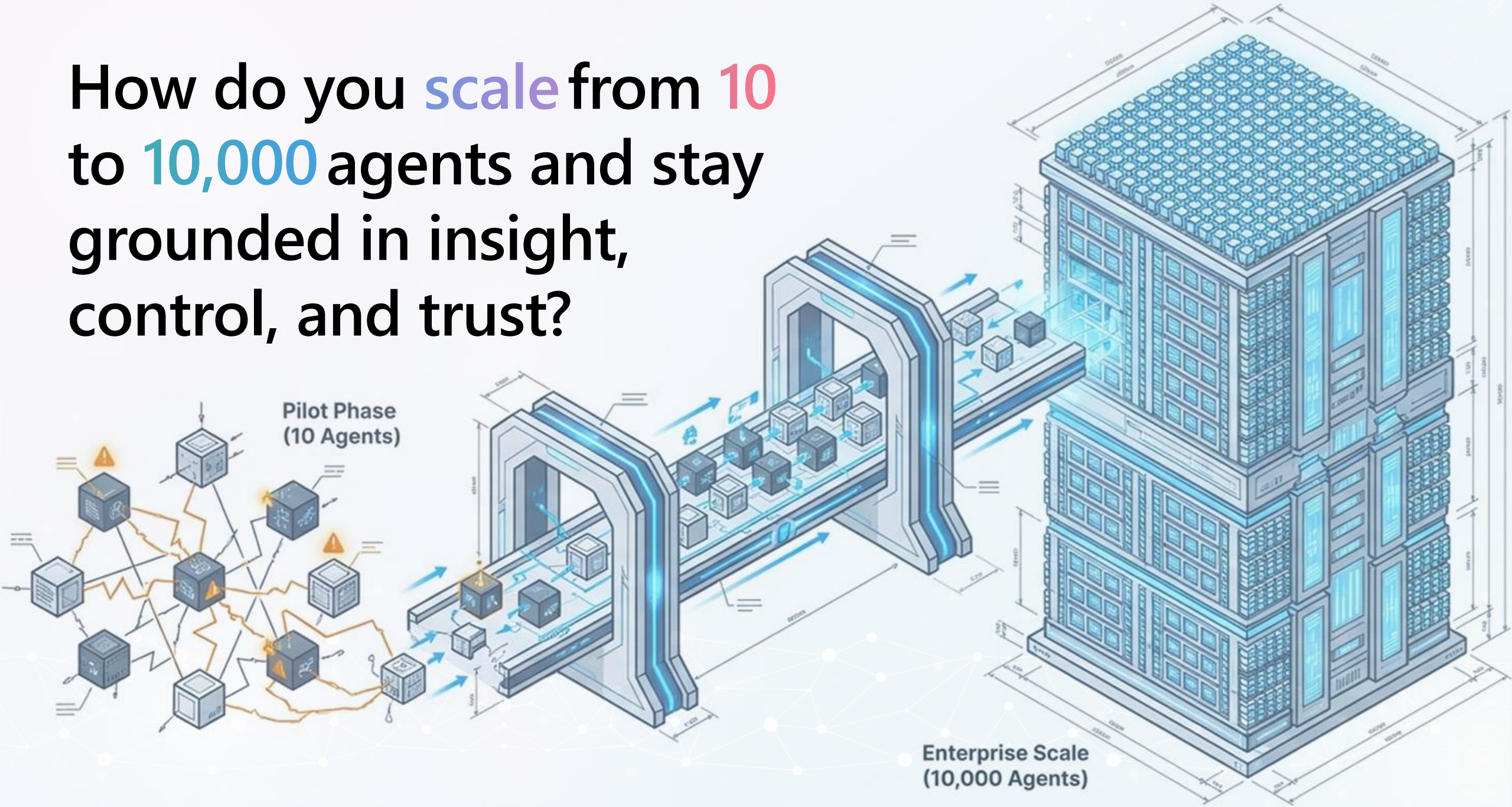
Governance vs. Velocity Paradox

Compliance traditionally slow down innovation

Trade-off is untenable

Enterprises can't afford to choose between speed and safety

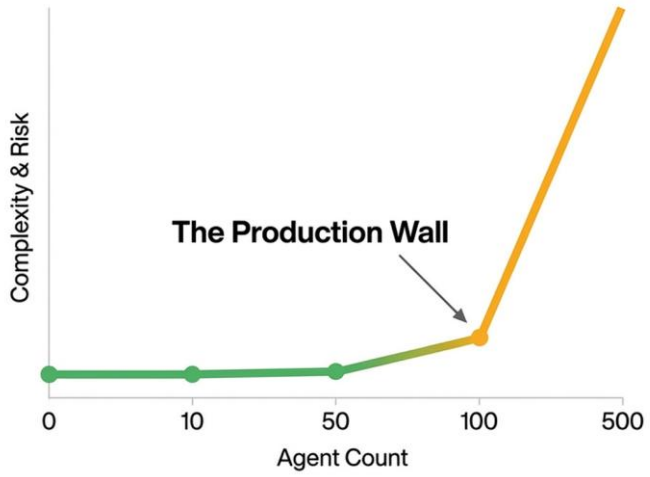
How do you scale from 10 to 10,000 agents and stay grounded in insight, control, and trust?



Why AI governance is the bridge to scale

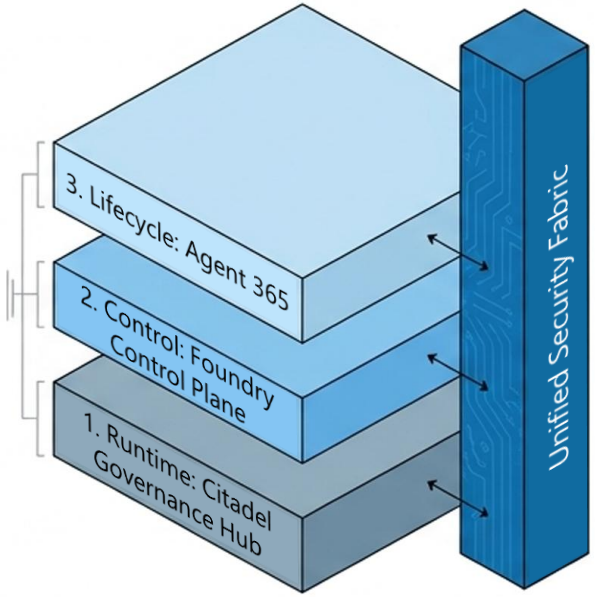
The Problem

As agents scale, risk, integration & cost grow exponentially → 95% of pilots fail to reach production



The Solution

Citadel Platform is a layered AI governance & security



The Proof

Microsoft named LEADER in Unified AI Governance

Source: https://www.microsoft.com/en-us/security/blog/2026/01/14/microsoft-named-a-leader-in-idc-marketscape-for-unified-ai-governance-platforms/

AI Citadel Platform – Bridging the gap

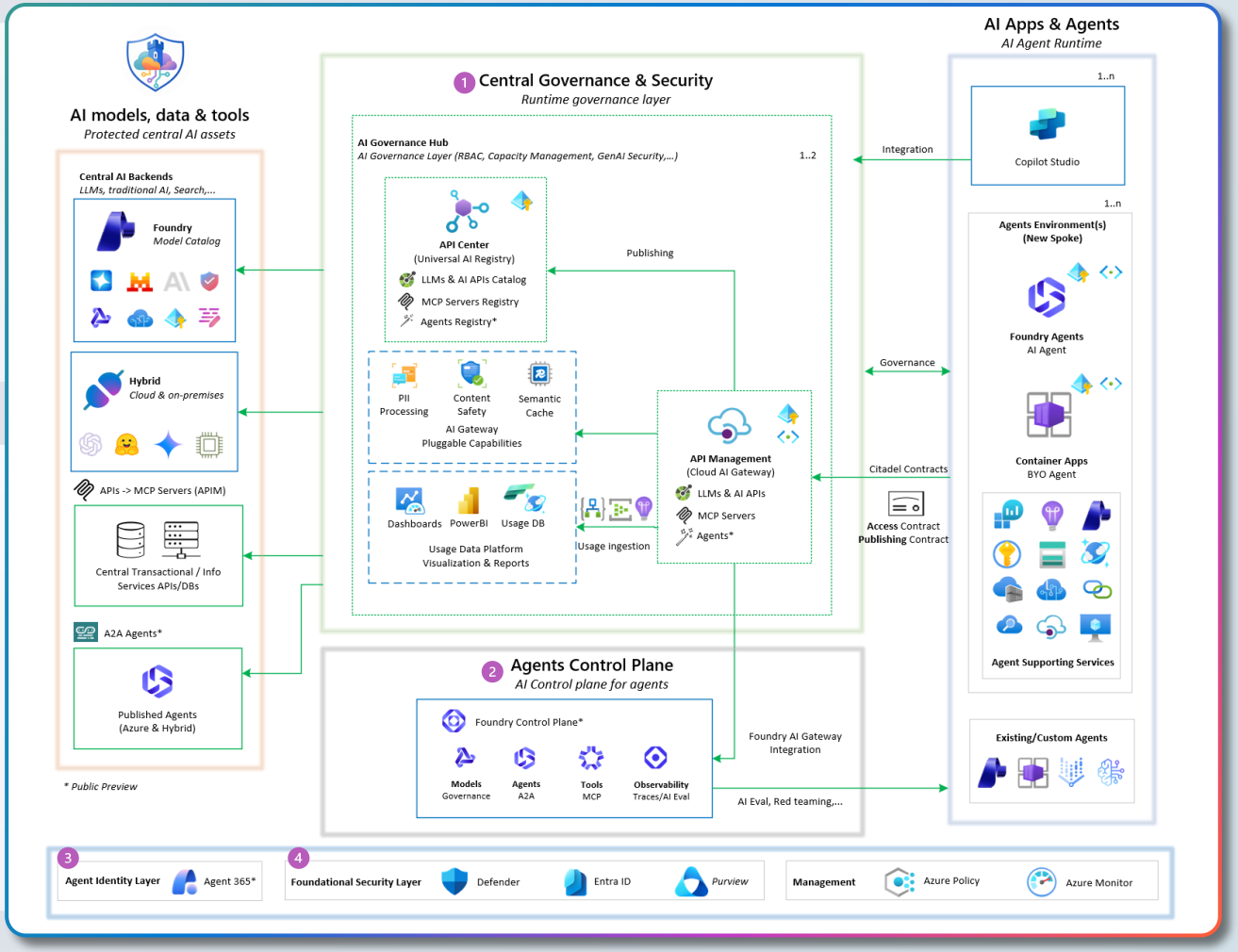
What it is

- AI Citadel Platform is a comprehensive guidance that bridges governance requirements and technical implementation.
- It's a layered end-to-end AI platform that organizations can trust to govern AI responsibly at speed

Key benefits

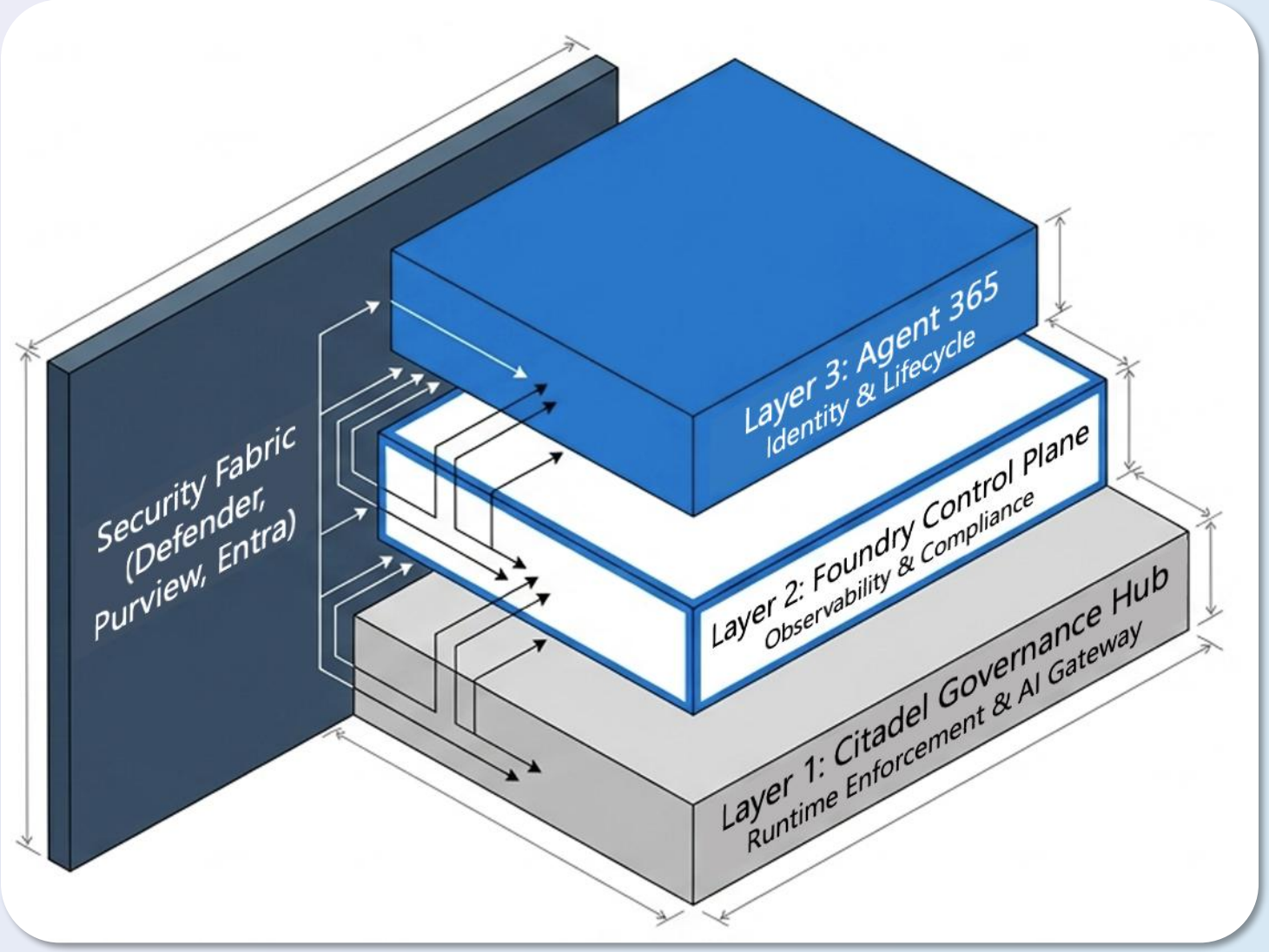
- Govern AI Responsibly with 4 governance & security layers (enforce policies on all AI usage)
- Maintain complete visibility (monitor & evaluate AI behaviour centrally + at agent level)
- Accelerate innovation (provide tools & templates for rapid, safe development)

Governance & Security | Observability & Compliance | AI Development Velocity



Citadel: Unified architecture for layered defense

Resolving the tension between speed and safety



Layer 1: Governance Hub

The runtime enforcement plane

What it is

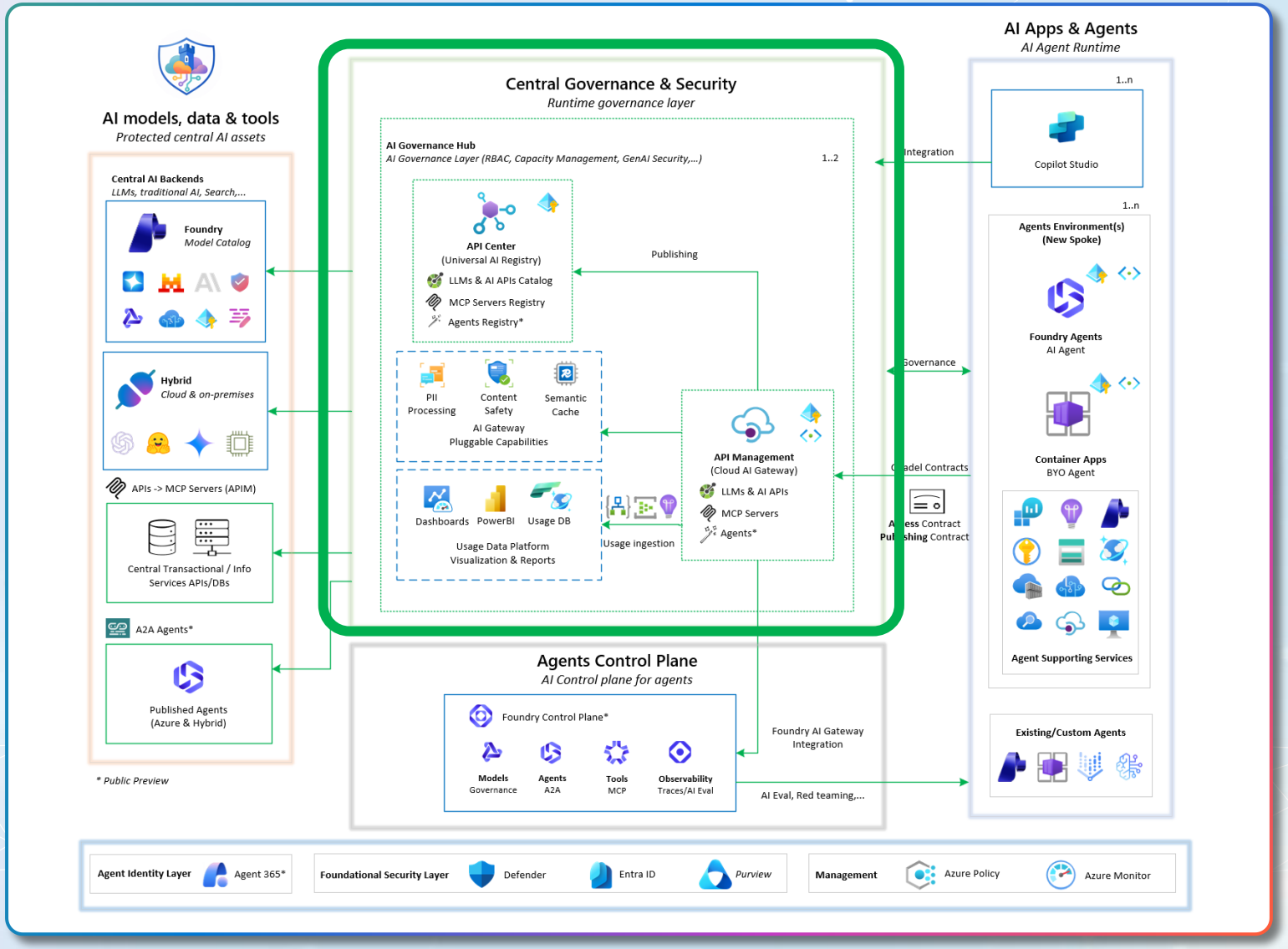
API Management
API Center

Core components

- Unified AI Gateway: single control point for models, tools and agents
- Universal registry: centralized catalogue for discovery

Enforcement capabilities

- Identity validation
- Access contracts
- Safety & guardrails enforcement



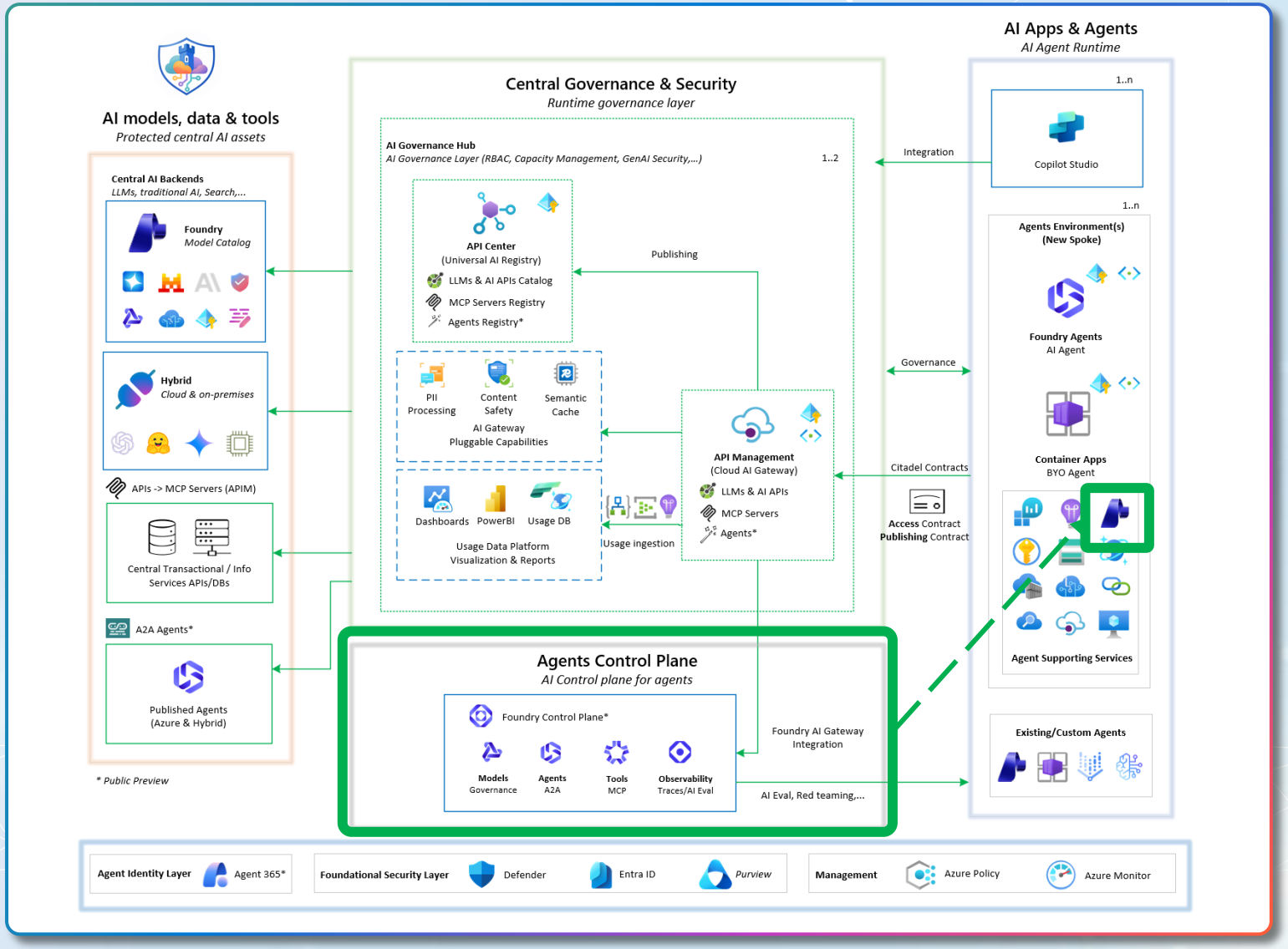
Layer 2: AI Control Plane

Automates trust so you don't have to manually review every interaction

What it is

Foundry Control Plane

- **Controls:** AI evaluations and compliance policies
- **Agent Traces:** end-to-end agent traces with AI Evaluations and metrics
- **Security:** Zero trust architecture, AI Red Teaming
- **Fleet ops:** track 100% of registered agents and manage activation



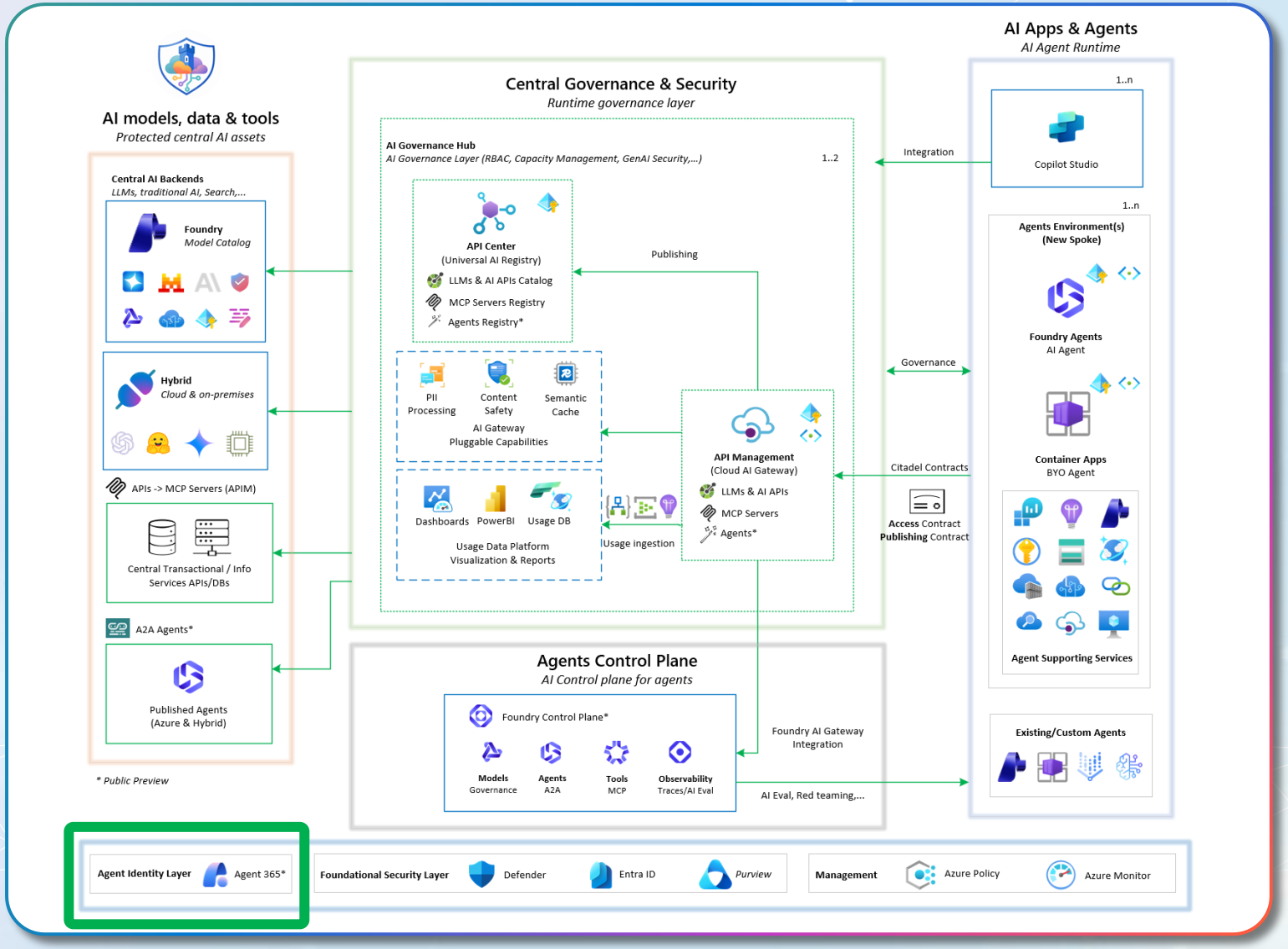
Layer 3: Agent Identity

Transforming agents to enterprise assets at scale

What it is

Agent 365

- Agent Identity Platform: where every published agents gets unique identity
- Detect shadow agents
- Ability to govern enterprise tools
- Managing agent's lifecycle and ownership



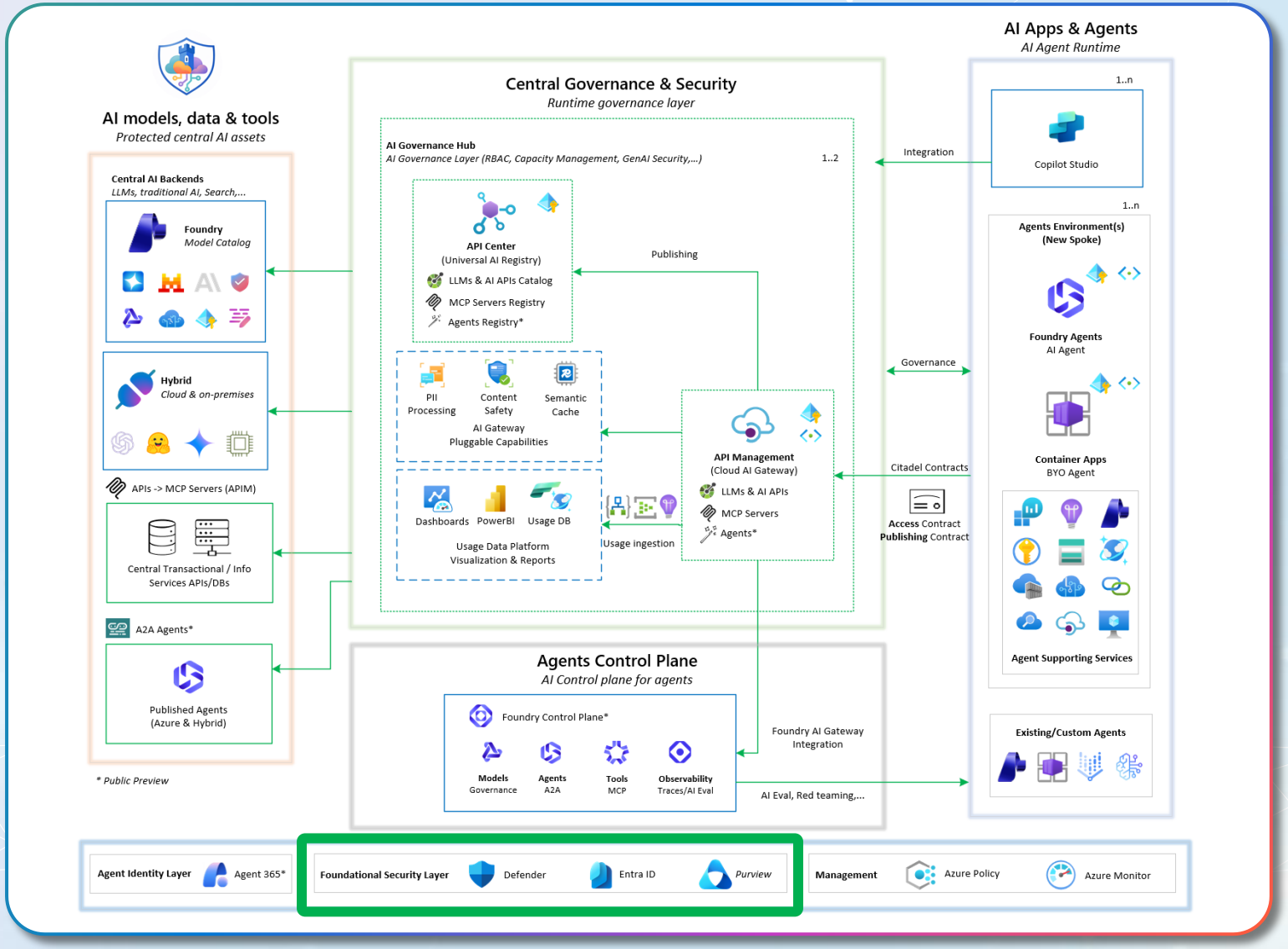
Layer 4: Security Fabric

Unified protection across all layers

What it is

Defender **Purview** **Entra**

- Microsoft Defender: Threat intelligence and AI security posture management
- Microsoft Purview: data governance, labelling and compliance
- Microsoft Entra: enterprise Agent and application identity platform for authentication and authorization



The Three Pillars of an Enterprise AI Platform

Governance & Security

Trustworthy AI Operations at Scale

- **Unified AI Gateway** for centralized control with central registry
- **Multi-cloud** and hybrid support
- **Agent identity** & lifecycle management
- **Security foundation** and threat protection

Observability & Compliance

End-to-End Monitoring, Evaluation & Trust

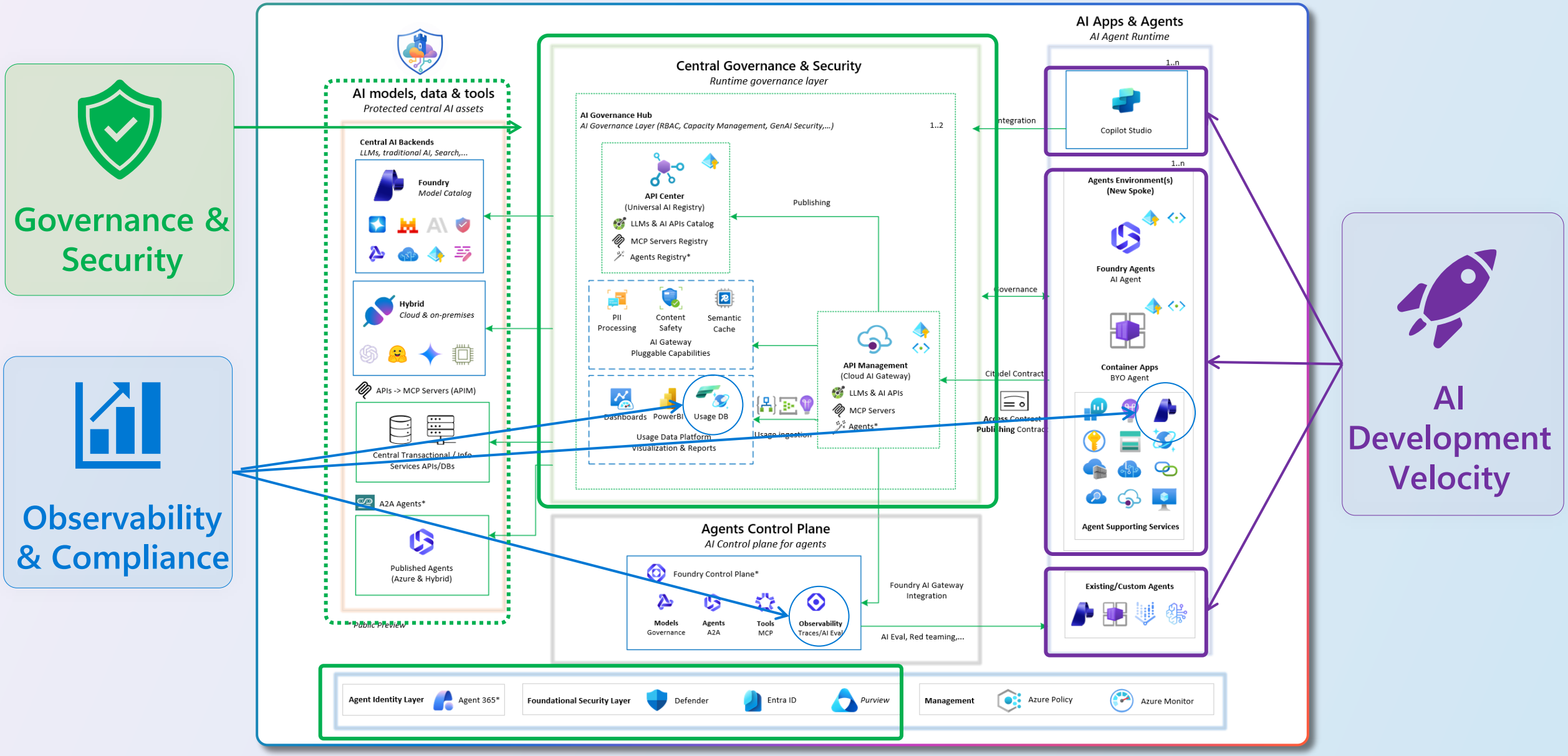
- **Platform-Level**: Centralized APM, usage tracking, automated evaluations, and enterprise alerting
- **Agent-Level**: Detailed execution traces, performance monitoring, and debugging tools
- **Rich Dashboards**: Integrated views for both operational and development teams

AI Development Velocity

Accelerating Innovation with Templates & Tools

- Pre-built deployment **templates**
- Flexible **agent development options** (low-code to pro-code)
- Citadel **AI Registry** for asset discovery and reuse
- **DevOps integration** for continuous delivery

Foundry Citadel Platform Architecture - Overview



Layer 1: Runtime Security – Hub-and-Spoke for Scale

AI Governance Hub

AI Hub Gateway Landing Zone

https://aka.ms/ai-hub-gateway

- Deployed **centrally**
  - With one deployment governing both Production and non-production
  - Or two deployments one for production and non-production (more can be added as per requirements)
  - Leverage Microsoft Foundry's AI agents, model catalog and AI evaluation

Access Contracts

Publish Contracts

Agents Environment

Foundry Landing Zone

https://github.com/Azure/AI-Landing-Zones

- Deployed **at a spoke**
  - Will have many deployments
  - This will be deployed for each business unit or use-case
  - Accelerate development through templated deployment that is configurable

Foundry Citadel Guidance: https://aka.ms/foundry-citadel

From Bottlenecks to Acceleration

Traditional Approach	Foundry Citadel Platform
❌ Manual risk assessments	✅ Automated compliance checks
❌ Scattered evaluation tools	✅ Unified observability platform
❌ Unclear requirements	✅ Codified governance contracts
❌ Implementation gaps	✅ Pre-built, proven patterns
❌ Friction between teams	✅ Streamlined collaboration
❌ Slow deployment cycles	✅ Rapid, repeatable deployments

Thank you
Q&A

Citadel Resources:
Citadel end-to-end guidance: https://aka.ms/foundry-citadel
Citadel Governance Hub (deployed centrally): https://aka.ms/ai-hub-gateway
Citadel Agents Environment (deployed many times): https://github.com/Azure/AI-Landing-Zones

Additional Resources:
https://aka.ms/cafai
https://aka.ms/wafai
https://azure.github.io/ai-app-templates/

Q&A

Replays Global AI Community

https://www.linkedin.com/company/global-ai-community-france/posts/?feedView=all

Microsoft Azure

Merci